

MARKETCAPTURE

Engineering Design and Rationale

Component-by-component design choices, alternatives, invariants, and failure behavior

Detailed reference for understanding, modifying, and defending the system

C++20 | NASDAQ ITCH 5.0 | MoldUDP64 | deterministic recovery | bounded-memory hot path

Design principles

- Determinism before throughput: never update a book across an unresolved sequence gap.
- Bounded memory: every queue, packet pool, reorder window, and arena has an explicit capacity.
- Single ownership: one writer owns each mutable structure.
- Evidence over claims: measurements identify host, compiler, configuration, and workload.
- Hardware adapters are separated from portable protocol and state logic.

1. Why C++20

C++ provides deterministic object lifetime, direct memory layout control, atomics, spans, PMR allocators, and access to kernel and DPDK APIs. C++20 adds `std::span`, atomic improvements, jthread-era concurrency facilities, and clearer compile-time constraints. The tradeoff is that lifetime and synchronization correctness are the developer's responsibility.

2. Why MoldUDP64

MoldUDP64 batches length-prefixed messages under a session and sequence header. Sequence identity enables duplicate suppression and gap detection, while batching amortizes network overhead. The decoder is bounds checked because a single malformed length could otherwise turn packet loss into memory corruption.

3. Why two parsers

ItchParser returns rich typed events with strings and is ideal for completeness, tools, and correctness comparisons. HotItchParser returns a fixed HotEvent and avoids string allocation on the order path. The independent golden decoder checks that optimization did not silently change protocol meaning.

4. Why fixed-width symbols

NASDAQ stock fields are fixed at eight bytes and MPID attribution is four bytes. Preserving those widths avoids heap allocation and makes queue elements predictable. Trimming into `std::string` occurs only when a cold consumer needs presentation or legacy serialization.

5. Why circular arbitration

A `std::map` naturally orders sequences but allocates nodes and adds pointer chasing. A modulo-indexed window has fixed memory and constant-time lookup. It is safe only while the configured reorder distance is respected; a collision is treated as capacity failure rather than overwrite.

6. Why SPSC queues

Each queue has one producer and one consumer, allowing head and tail cursors without locks or compare-and-swap loops. Producer release publishes a completed slot; consumer acquire observes it. Cache-line alignment reduces false sharing. SPSC is deliberately not advertised as MPMC.

7. Why value fan-out

Each downstream worker receives an owned HotEvent. This prevents a slow recorder from retaining the NIC packet and keeps lifetime reasoning simple. The cost is a small fixed copy, chosen over reference counting and cross-thread reclamation complexity.

8. Why an arena plus recycling pool

General-purpose allocation adds latency variance and may lock internally. A fixed arena bounds memory and a PMR pool reuses erased hash nodes during order churn. `null_memory_resource` prevents hidden heap fallback. Capacity planning is therefore an operational requirement.

9. Why a global order index

Execute, cancel, delete, and replace messages identify an order reference rather than a symbol. A global order map gives constant-time routing to symbol, side, price, and remaining shares. Price-level keys aggregate total shares and order count.

10. Why asynchronous recording

Disk write, compression, page faults, and fsync have long-tail latency. Recorder and PCAP workers isolate those effects from RX and parsing. Separate queues also expose storage lag through watermarks instead of contaminating packet latency invisibly.

11. Why two checkpoint generations

An in-place checkpoint can be torn by power loss. Alternating slots retain a previous valid generation. Checksums detect corruption; fsync and atomic rename establish durability ordering. The design favors simple auditable recovery over a complex transactional database.

12. Why PCAP plus normalized ticks

PCAP preserves exact ingress bytes for protocol debugging and recovery. Normalized ticks are smaller and convenient for deterministic book replay. Keeping both separates wire evidence from application representation.

13. Why recvmmsg, io_uring, and DPDK

recvmmsg is a simple batched kernel path. Multishot io_uring reduces submission overhead and uses provided buffers. DPDK bypasses the kernel and is appropriate when NIC ownership, huge pages, and dedicated cores are available. They are alternatives selected by deployment evidence.

14. Why thread affinity

Pinning prevents migration-related cache disruption and makes measurements reproducible. It is optional because incorrect pinning can compete with NIC IRQs or place memory on the wrong NUMA node. CPU IDs are configuration, not hard-coded policy.

15. Why explicit overload policies

Reject protects the RX thread and exposes loss immediately. Spin preserves work only if upstream behavior and CPU budget allow backpressure. Unbounded queues are rejected because they convert overload into eventual memory failure and enormous tail latency.

16. Why percentiles and watermarks

Mean latency hides rare stalls. p50 describes the center; p99 and p99.9 reveal tail behavior. Queue watermarks identify accumulating work. NIC missed packets, application rejects, gaps, parse errors, and storage lag must be interpreted together.

17. Why fuzzers and sanitizers

Binary decoders face adversarial lengths and truncation. Prefix tests, libFuzzer, ASan, and UBSan cover different failure classes. They do not prove correctness, so golden and official-file tests provide independent semantic evidence.

18. Why deterministic simulation

A seeded simulator generates reproducible order lifecycles and A/B loss patterns. It makes bugs repeatable and CI friendly. It is not a substitute for independently produced NASDAQ data, which is why the BinaryFile validator exists.

19. Hardware boundaries

SO_TIMESTAMPING, DPDK, and the FPGA DMA ABI are real adapters, but compile success is not physical performance proof. Line rate, timestamp accuracy, NUMA behavior, and DMA coherency must be measured on the selected NIC, clock, driver, and FPGA.

20. Security and operations

Feed configuration contains network assignments, not portal credentials. Capture files may contain commercially controlled data. Production service management must cover permissions, disk alarms, retention export, log rotation, restart policy, and secret handling outside source control.

Component contract matrix

Component	Invariant	Failure signal
Packet pool	One submitter; slot returned after parse	submit=false / reject counter
Arbitrator	Only expected sequence is delivered	recovery request / window error
Hot parser	Exact type and length	ParseError
SPSC	Exactly one producer and consumer	full queue / watermark
Arena book	Order and level totals agree	capacity or transition exception
Recorder	Only order-impacting events	worker exception
Checkpoint	Newest checksum-valid generation	fallback or no-valid-generation

Performance interpretation

A benchmark number is valid only with its CPU model, kernel, compiler, optimization flags, affinity, NUMA policy, governor, workload size, queue policy, active-order population, and recorder configuration. A faster synthetic parser number does not prove end-to-end market-feed capacity.

Modification checklist

- State the invariant the change preserves.
- Add malformed, capacity, shutdown, and replay tests.
- Run sanitizers and fuzz smoke.
- Measure same-host before and after distributions.
- Inspect flamegraph and queue watermarks.
- Document any new deployment assumption or external dependency.